

MVC Architecture Patterns in Java: Spring MVC and Jakarta EE MVC (Jakarta MVC) — Implementation, Best Practices, Database Integration, and Real-World Examples

Executive Summary

Model–View–Controller (MVC) remains the dominant architectural pattern for building server-rendered web applications in Java. Its enduring appeal lies in the clear separation of concerns: user interface logic resides in the view layer, domain behavior and state in the model, and request handling and coordination in the controller. This separation improves maintainability, modularity, and testability, enabling teams to evolve components independently and manage complexity at enterprise scale.^[^1]

Two mature approaches to MVC are prevalent in today’s Java ecosystem:

- Spring MVC, the de facto mainstream framework, built atop the servlet container and organized around the Front Controller pattern implemented by DispatcherServlet. It offers a flexible component model, robust view resolution (JSP, Thymeleaf, and others), and tight integration with the broader Spring ecosystem for data access, security, and transactions.^{[^2][^3]}
- Jakarta MVC (formerly MVC 1.0 under JSR 371), standardized within Jakarta EE and built on Jakarta RESTful Web Services (JAX-RS). It reuses JAX-RS for binding and mapping while introducing MVC-specific semantics for controllers, views, and models, plus web-focused features such as form binding with BindingResult, redirect-scoped beans, first-class CSRF protections, and events.^{[^4][^5]}

For modern Java web development, our recommendations are as follows:

- Choose Spring MVC when you want a mature, ubiquitous framework with deep ecosystem integrations, established testing and DevOps practices, and broad community support. It excels when you need both server-rendered pages and JSON APIs and when leveraging Spring Boot is a priority.^[^3]
- Choose Jakarta MVC when you are committed to the Jakarta EE platform and prefer a standards-based action-oriented MVC built directly on JAX-RS. It is well-suited to environments that favor platform standardization and portable behavior across compliant servers.^{[^4][^5]}

Key pitfalls to avoid:

- Treating the controller as a dumping ground for business logic, which undermines separation of concerns and testability.^[^1]

- Exposing JPA entities directly to views or external APIs, which leaks persistence concerns and sensitive data. Use DTOs and projections.^[^15]
- Over-fetching or under-controlling transactions, leading to N+1 queries or inconsistent state; prefer explicit fetch strategies (@EntityGraph) and transactional boundaries in services.^{[^15][^16]}
- Inadequate CSRF protections for browser-based applications; follow platform guidance (synchronizer tokens, SameSite cookies as defense-in-depth).^[^6]

This guide distills actionable practices for both frameworks, highlights database integration patterns and testing strategies, and provides case-study-backed implementation guidance.

MVC Fundamentals and Evolution in Java

At its core, MVC partitions application responsibilities into three interoperating components:

- Model: Domain data, state, and business behavior.
- View: Presentation logic and rendering; how the model is displayed to the user.
- Controller: Receives input, coordinates with the model (often via services), selects the view, and assembles the model for rendering.

By enforcing these boundaries, MVC improves maintainability and testability and allows views to evolve without entangling business logic, and vice versa.^[^1]

In the Java ecosystem, MVC evolved along two distinct lines:

- Servlet-centric, action-based frameworks (e.g., Spring MVC). The front controller centralizes request handling, then delegates to controller methods that prepare models and select views.
- JAX-RS-centric, action-based MVC (Jakarta MVC). Controllers are JAX-RS resource methods marked by @Controller, with view selection and model handling specified by the MVC standard.

Both approaches preserve the spirit of MVC while leveraging platform capabilities. The separation of concerns remains the compass: keep business logic in services and the model, keep request handling and view coordination in controllers, and keep presentation in views.^[^1]

Spring MVC Architecture

Spring MVC implements the Front Controller pattern via DispatcherServlet, which orchestrates the request flow through handler mappings, adapters, controllers, view resolvers, and interceptors. This composition yields a flexible and extensible request-handling pipeline.^{[^2][^3]}

To orient the discussion, Table 1 maps core components to their responsibilities.

Table 1 — Spring MVC core components and responsibilities

Component	Role in the Request Flow
DispatcherServlet	Central front controller that intercepts requests and coordinates the MVC pipeline.
HandlerMapping	Resolves which controller/handler should process the request (e.g., RequestMappingHandlerMapping).
HandlerExecutionChain	Wraps the handler with configured interceptors for pre/post processing.
HandlerAdapter	Adapts the handler's invocation model to the servlet contract; constructs ModelAndView.
Controller	Handles input, collaborates with services, populates model, returns view name or JSON.
ModelAndView	Container for model data and logical view name returned to DispatcherServlet.
ViewResolver	Resolves logical view names to concrete View implementations for rendering.
Interceptors	Cross-cutting request/response processing (e.g., locale, logging, security).

The flow is deliberately modular: DispatcherServlet delegates strategy selection (mapping, adapter) and composition (interceptors) so controller code remains focused on application logic and the view layer remains decoupled from request handling.^[2]^[3]

DispatcherServlet and Request Flow

DispatcherServlet is the traffic cop for Spring MVC:

- It consults registered HandlerMapping beans to locate a suitable handler and its interceptors.
- It selects a HandlerAdapter capable of invoking that handler (for annotated controllers, typically RequestMappingHandlerAdapter).
- The controller method executes within a prepared model-and-view context; for REST endpoints, responses are written directly without view resolution.
- DispatcherServlet then resolves the view via ViewResolver and renders the model, or, in the REST case, serializes the response body.^[2]^[3]

This pipeline allows concerns like security checks, internationalization, and exception translation to be applied uniformly at the front door rather than scattered across controllers.^[2]

Implementation Strategies

- Prefer annotation-based controllers with `@RequestMapping` and its shortcuts (`@GetMapping`, `@PostMapping`). They provide clear, idiomatic routing and data binding semantics.^[^3]
- Use view resolvers for server-rendered pages (JSP, Thymeleaf). For APIs, prefer `@RestController` or `@ResponseBody` to bypass view resolution and write serialized payloads directly.^[^3]
- Employ interceptors to apply cross-cutting behaviors uniformly. For example, a locale interceptor can resolve the request locale for i18n without polluting controller logic.^{[^2][^3]}

These choices keep controllers lean, testable, and aligned with Spring' s conventions.

Jakarta EE MVC (Jakarta MVC) Architecture

Jakarta MVC 2.0 is a standard action-oriented MVC framework built on JAX-RS, leveraging CDI and Bean Validation. Controllers are JAX-RS resource methods annotated with `@Controller`. Views and models are handled per the MVC spec, with required support for JSP and Facelets as view technologies.^{[^4][^5]}

Key capabilities include:

- Controller definition and view selection rules, including void-return controllers decorated with `@View`.
- Two model types: CDI `@Named` beans and the Models map interface.
- Binding and validation that remain controller-friendly through `@MvcBinding` and `BindingResult`, enabling form error handling without exceptions.
- CSRF protection via `@CsrfProtected` and `MvcContext`.
- Redirect-scoped beans for the Post-Redirect-Get (PRG) pattern.
- Event hooks for lifecycle observability (`BeforeControllerEvent`, `AfterControllerEvent`, etc.).^{[^4][^5][^6]}

Table 2 summarizes Jakarta MVC' s annotations.

Table 2 — Jakarta MVC annotations summary

Annotation	Target	Purpose
<code>@Controller</code>	Type/method	Marks a JAX-RS resource method as an MVC controller; can be class- or method-level.
<code>@View</code>	Type/method	Declares the view for void-return controller methods; can set a default at class-level.
<code>@MvcBinding</code>	Field/method/parameter	Enables MVC binding/validation behavior, allowing controller invocation even with errors.

Annotation	Target	Purpose
@RedirectScoped	Type/method/ field	Extends a bean's lifetime across a redirect for PRG workflows.
@UriRef	Method	Assigns a stable symbolic name to a controller method for URI generation in views.
@CsrfProtected	Method	Requires CSRF token validation for the annotated controller method.

The spec also prescribes behaviors for redirects (303 recommended), CSRF token conveyance (headers or hidden fields), and locale resolution strategies to support i18n in data binding and view formatting.^{[4][6]}

Controllers and Views

Controller methods may return void (requiring @View), a String interpreted as a view path, or a JAX-RS Response whose entity is a view path or other permitted type. Hybrid classes can mix JAX-RS resources and MVC controllers in one codebase.^{[4][6]}

View engines are discovered via CDI. The runtime selects the highest-priority engine that supports the view (built-in JSP and Facelets are mandatory). Model entries are bound to the request so Expression Language (EL) can access them in views. Relative view paths are recommended for safety and maintainability.^[4]

Data Binding, Validation, and Error Handling

Jakarta MVC extends JAX-RS binding with @MvcBinding. When binding or validation errors occur, the controller is still invoked, and errors can be inspected via injected BindingResult. Controllers must check BindingResult and react accordingly (e.g., re-render the form with errors), avoiding exception-driven flows that derail the user journey.^{[4][6]}

Numeric and boolean conversions are locale-aware under @MvcBinding, which is crucial for internationalized applications. The specification defines concrete rules for parsing numeric types and booleans; other conversions are implementation-specific.^[4]

Security: CSRF and XSS

The MVC spec provides CSRF protections via @CsrfProtected, a configurable CsrfProtection property, and access to a CSRF token in the MvcContext (and EL). Tokens can be conveyed as form fields or headers. Validation failures result in a 403 status, and implementations must support validation for POSTed form submissions.^[4]

Defense against cross-site scripting (XSS) is addressed via contextual encoders obtained from MvcContext, encouraging proper escaping in views. This complements CSRF defenses to cover common web application vulnerabilities.^[4]

Jakarta MVC builds on JAX-RS for mapping and binding, CDI for component lifecycle, and Bean Validation for constraints, aligning MVC with the broader Jakarta EE platform capabilities.^{[^5][^6]}

Database Integration Patterns

Most MVC applications require robust persistence with consistent transaction management and careful control over data access patterns. Two mainstream paths exist:

- Spring Data JPA on top of JPA/Hibernate, with Spring’s `@Transactional` declarative transactions and rich repository abstractions.^{[^7][^8][^9]}
- Direct JPA usage within Jakarta MVC applications, leveraging JPA, JDBC, or other data access strategies under the Jakarta EE umbrella (with transactions coordinated by the container or JTA as appropriate). While the MVC spec itself defines MVC behavior, data access patterns follow general Jakarta EE practices beyond the spec’s scope.^{[^4][^5]}

Beyond choosing an ORM, architecture decisions should center on layering and boundaries. Controllers should coordinate with services, and services should own transactional boundaries. Repositories (or DAOs) encapsulate persistence concerns and expose query APIs. This keeps controllers focused on web coordination, maximizes testability, and respects separation of concerns.^[^1]

Table 3 introduces typical layers and technology choices.

Table 3 — Layering and technology mapping

Layer	Primary Responsibilities	Technologies/Practices
Controller	Handle input, select views, assemble models	Spring MVC <code>@Controller</code> or Jakarta MVC <code>@Controller</code>
Service	Business logic, orchestration, transactional boundaries	POJO services; <code>@Transactional</code> (Spring); container-managed or JTA (Jakarta EE)
Repository/DAO	Data access, queries, persistence semantics	Spring Data JPA/JPA/Hibernate; JDBC; (Jakarta EE: JPA, JDBC)

ORM workflows fall into two broad patterns: database-first and code-first. Each suits different organizational contexts and constraints.

Table 4 — ORM workflows comparison

Aspect	Database-First	Code-First
Source of truth	Existing schema (tables, constraints, relationships)	JPA entity classes define the model

Aspect	Database-First	Code-First
Tooling	Generate entities from schema; configure persistence	Generate/演化 schema via hbm2ddl or migrations (Flyway/Liquibase)
Strengths	Aligns with legacy schema; leverages stored procedures	Rapid iteration; tight code-schema alignment
Considerations	Validate generated entities; map complex features explicitly	Test schema generation; govern migrations rigorously

Both workflows benefit from careful error handling, secure connection management, and dependency injection for the persistence unit.^[^22]

Spring Data JPA Best Practices

- Use DTOs instead of entities in API responses. Convert entities to DTOs in services or via projections to avoid leaking persistence state and to tailor payloads. Automated mapping libraries (e.g., MapStruct) can reduce boilerplate.^[^15]
- Prefer lazy loading (FetchType.LAZY) with explicit fetching strategies to avoid N+1 problems. Use @EntityGraph to load required relationships in a single query when needed.^[^15]
- Apply @Modifying with @Transactional for bulk updates and deletes. Keep transaction boundaries in services and consider clearAutomatically when the persistence context must be reset.^[^15]
- Use pagination for large datasets to bound memory and latency; consider keyset pagination for very large tables.^[^15]
- Prefer @Query (JPQL or native) for complex queries and aggregations; name parameters and profile indexing to ensure performance.^[^15]
- Rely on connection pooling (HikariCP is the Spring Boot default) and right-size pool settings for workload.^[^15]
- Enable optimistic locking via @Version to prevent lost updates under concurrency.^[^15]

These practices improve performance, security, and maintainability without sacrificing transactional integrity.

Transaction Management Essentials (Spring)

Spring' s declarative model via @Transactional simplifies transaction scoping while preserving control. Key concepts include:

- Physical vs logical transactions and propagation levels (e.g., REQUIRED, REQUIRES_NEW, NESTED) govern how interactions behave within existing transactions.

- Isolation levels control visibility and interference among concurrent transactions; nested transactions may rely on savepoints.
- Proxies (CGLib or JDK dynamic proxies) intercept `@Transactional` method calls, delegating to a `PlatformTransactionManager` (`JpaTransactionManager`, `HibernateTransactionManager`, or `DataSourceTransactionManager`).^[^16]

Service layers should own transactional boundaries; controllers should not annotate themselves unless a specific controller-level boundary is warranted. Understanding propagation and isolation avoids subtle anomalies and performance cliffs.^[^16]

Security in MVC Applications

For browser-accessible applications, CSRF is a foundational risk. Spring Security provides a comprehensive synchronizer token pattern and encourages defense-in-depth via cookie attributes such as `SameSite`.^[^6]

Jakarta MVC provides first-class CSRF protections via `@CsrfProtected`, token access through `MvcContext`, and configurable enforcement modes.^[^4]

Table 5 contrasts these mechanisms.

Table 5 — CSRF protection comparison

Capability	Spring Security	Jakarta MVC
Token pattern	Synchronizer token pattern (hidden field or header)	CSRF token accessible via <code>MvcContext</code> ; tokens as fields/headers
Enablement	Enabled by default for unsafe methods; configurable	<code>@CsrfProtected</code> for explicit enforcement; configurable <code>CsrfProtection</code>
Failure handling	Reject request; 403 on mismatch	<code>CsrfValidationException</code> ; default 403 mapping
Defense-in-depth	<code>SameSite</code> cookie attribute via Spring Session/Resolution	N/A (platform-level; use container/session features)

Additional recommendations:

- Ensure login and logout endpoints enforce CSRF and account for session timeouts in user experience.
- For multipart file uploads, include the token in the body to avoid unauthorized temporary uploads; when unacceptable, include it in the URL as a fallback.^[^6]
- For stateless APIs that rely on custom headers or token cookies, remember that browser-automatic credentials still constitute a CSRF risk if the server does not validate request-specific tokens.^[^6]

Testing Strategies

Testing MVC applications entails layered verification:

- Controller/unit tests assert routing, binding, and view selection (or JSON serialization).
- Service tests focus on business logic and transactional behavior.
- Repository tests validate persistence semantics and query correctness.

Spring MVC Test (MockMvc) is the standard tool for testing the web layer without a running servlet container, enabling assertions on view names, model attributes, and JSON payloads.^{[17][18]}

Table 6 provides a concise testing matrix.

Table 6 — Testing tools and scope

Layer	Primary Tools	Scope and Focus
Web	Spring MVC Test (MockMvc)	Routing, binding, view selection, JSON responses
Service	JUnit + Spring Test (@ExtendWith, @Transactional)	Business logic, orchestration, transaction propagation
Repository	JPA Test frameworks, Testcontainers, in-memory DB	Query correctness, CRUD semantics, transaction integration

MockMvc supports rich request builders and expectations (status, view, content), path variables, query parameters, and form posts. It can print request/response details for debugging and is well suited to continuous integration pipelines.^{[17][18]}

Real-World Examples and Case Studies

Industry-scale applications underscore the value of disciplined MVC layering and data access practices:

- Large-scale social networking and e-commerce systems process millions of events per second on Java backends, relying on layered architectures (including MVC) for resilience and maintainability.^[19]
- Case studies highlight MVC’ s role in streamlining web interfaces and separating UI from domain logic, with ORM, caching, and asynchronous processing as complementary enablers.^{[20][21]}
- Enterprise patterns emphasize layered responsibilities and pragmatic data access (DAO, Repository) to balance performance and simplicity in complex systems.^{[19][21]}

For concrete, reusable patterns:

- The Spring Boot RealWorld example demonstrates a CRUD application with authentication, structured layering, and RESTful APIs—useful as a reference for repository/service/controller composition and security integration.^[^20]
- An enterprise REST API example shows Spring MVC binding, Spring Data JPA/Hibernate integration, and secured admin endpoints, illustrating modern practices for building admin panels and APIs side by side.^[^21]

These examples reinforce that successful projects combine MVC separation-of-concerns with disciplined data access, transactions, and testing.

Comparative Analysis: Spring MVC vs Jakarta MVC

Both frameworks deliver action-oriented MVC, but they differ in platform integration and defaults.

Table 7 — Side-by-side comparison

Dimension	Spring MVC	Jakarta MVC
Programming model	DispatcherServlet + HandlerMapping/Adapter + annotations	JAX-RS resource methods + @Controller + view selection rules
View technologies	JSP, Thymeleaf, others via ViewResolver	JSP and Facelets (mandatory support), extensible via CDI
Data binding & validation	Spring binding and validation (platform-agnostic)	JAX-RS binding extended with @MvcBinding + BindingResult
Security	Spring Security (CSRF, authN/Z, session features)	@CsrfProtected, MvcContext CSRF tokens; JAX-RS/security integration
Transactions	@Transactional (PlatformTransactionManager)	Container-managed/JTA (outside MVC spec; per Jakarta EE patterns)
Event model	Interceptors and handler lifecycle; no spec-defined MVC events	CDI MVC events (Before/AfterController, Before/AfterProcessView, ControllerRedirect)
Testing	Spring MVC Test (MockMvc), Spring Test ecosystem	JAX-RS testing patterns (e.g., Jersey test container), CDI tests
Ecosystem maturity	Very broad (Spring Boot, Data, Security, Cloud)	Standardized spec (Jakarta EE), implementations (e.g., Krazo)

Dimension	Spring MVC	Jakarta MVC
Ecosystem metrics	Scores high in documentation/ community per comparative reviews	Scores well for standardization and spec compliance

In comparative reviews, Spring’ s strengths include its documentation depth and ecosystem breadth. Jakarta MVC’ s strengths include standardization and platform alignment. Selection should reflect organizational priorities: ecosystem integration and speed (Spring) vs platform standardization and portability (Jakarta).^[24]^[4]^[3]

Modernization and Migration Strategies

Many organizations seek to modernize legacy Java EE platforms, including EJB-based systems, by adopting Spring Boot and cloud-native operations. A refactoring-first approach is recommended:

- Adopt the Strangler Fig pattern: incrementally replace functionality while systems remain operational. Modernize transactions (from EJB CMT to Spring @Transactional or JTA) and peel away vendor lock-in by migrating to embedded servers and container deployments.^[25]
- Inventory dependencies early, prioritize stateless modules, and generate regression tests before changes to ensure parity. Canary releases, observability, and rollback plans are essential.^[25]

Table 8 — Migration checklist (illustrative)

Step	Action	Pro Tip
Assess & Plan	Inventory EJBs, JNDI, JMS, data sources, deployment descriptors	Prototype critical modules first
Transaction Modernization	Move to Spring transactions or JTA	Measure throughput before/after
Refactor Services/Components	Extract services; replace EJBs; decouple UI	Start with stateless modules
Data Layer Refactoring	Harden JPA/Hibernate patterns; align with Spring Boot	Replace proprietary services (e.g., vendor JMS)
Security Modernization	Migrate to Spring Security/OAuth2	Bake in SSO where applicable
Testing & Parity	Auto-generate integration and regression tests	Monitor for behavior drift
Deployment & Rollout	Containerize; canary; observability; rollback plan	Feature flags and traffic splitting

A stepwise migration lowers risk while increasing developer productivity through embedded containers, CI/CD alignment, and cloud readiness.^[^25]

DevOps and Performance Tuning for MVC Applications

High-traffic Java applications require deep instrumentation and pragmatic JVM tuning:

- Instrument JVM metrics (GC pauses, heap/non-heap usage), thread pools, database connection pools, and queue sizes. Visualize via Prometheus/Grafana or APM tools.^[^26]
- Tune JVM parameters in context: right-size heap, select appropriate GC algorithms, and profile under realistic load. Avoid one-size-fits-all configurations.^{[^26][^27]}
- Apply application-level optimizations: caching (Redis/memcached), load balancing, query tuning, indexing, CDNs, and asynchronous processing for throughput and latency improvements.^{[^19][^27]}

These operational practices complement architectural separation of concerns by ensuring the running system remains observable and responsive under stress.

Recommendations and Decision Matrix

- If your organization prioritizes rapid development, vast ecosystem integrations, and readily available talent, choose Spring MVC. It offers mature tooling for data access, transactions, security, testing, and DevOps workflows.^[^24]
- If your organization prioritizes platform standardization, portability across compliant servers, and JAX-RS alignment, choose Jakarta MVC. It provides a clean action-oriented MVC with web-focused features integrated into Jakarta EE.^{[^4][^5]}

Table 9 — Decision matrix (team, ecosystem, constraints → recommended stack)

Criterion	Indicators	Recommended Stack
Ecosystem integration needs	Heavy reliance on Spring Boot, Spring Cloud, Spring Security	Spring MVC + Spring ecosystem
Platform standardization	Need for portable behavior across Jakarta EE servers	Jakarta MVC (JAX-RS + CDI + MVC)
Testing and DevOps tooling	Strong requirement for MockMvc and CI/CD familiarity	Spring MVC
Developer availability	More developers experienced with Spring	Spring MVC

Criterion	Indicators	Recommended Stack
Legacy Java EE modernization	EJB/JMS/JNDI in legacy stack; desire to containerize	Spring MVC (modernize via Boot)

Whichever path you choose, keep the controller thin, the service layer robust, and the data access layer explicit. Use DTOs to decouple persistence from representation, and secure browser-facing endpoints with appropriate CSRF defenses.

Information Gaps

- No head-to-head performance benchmarks between Spring MVC and Jakarta MVC under identical workloads were available in the reviewed sources.
- Jakarta MVC production case studies are less abundant than Spring MVC's; direct verifiable examples are limited.
- A fully end-to-end Jakarta MVC project with persistence and tests was not included in the collected examples.
- Detailed migration playbooks from Jakarta MVC to Spring MVC (as opposed to general Java EE-to-Spring Boot) were not found.

These gaps suggest practical next steps: conduct in-house performance tests, prototype Jakarta MVC in a representative microservice, and capture internal case studies to inform future decisions.

References

- [^1]: Model-View-Controller Pattern in Java (Java Design Patterns). <https://java-design-patterns.com/patterns/model-view-controller/>
- [^2]: Introduction to Spring MVC Architecture (Javalaunchpad). <https://javalaunchpad.com/introduction-to-spring-mvc-architecture/>
- [^3]: Spring MVC Tutorial (Baeldung). <https://www.baeldung.com/spring-mvc-tutorial>
- [^4]: Jakarta MVC Specification 2.0. <https://jakarta.ee/specifications/mvc/2.0/jakarta-mvc-spec-2.0>
- [^5]: JSR 371: MVC 1.0 (Java Community Process). <https://jcp.org/en/jsr/detail?id=371>
- [^6]: Cross Site Request Forgery (CSRF) - Spring Security Reference. <https://docs.spring.io/spring-security/reference/features/exploits/csrf.html>
- [^7]: Getting Started — Accessing Data with JPA (Spring). <https://spring.io/guides/gs/accessing-data-jpa>
- [^8]: Spring MVC + Spring Data JPA + Hibernate + JSP + MySQL Tutorial (Java Guides). <https://www.javaguides.net/2018/11/spring-mvc-5-spring-data-jpa-hibernate-jsp-mysql-tutorial.html>

[^9]: Transactions with Spring and JPA (Baeldung). <https://www.baeldung.com/transaction-configuration-with-jpa-and-spring>

[^10]: Spring Framework Reference: Declarative Transaction Management (5.3.x). <https://docs.spring.io/spring-framework/docs/5.3.x/reference/html/data-access.html#transaction-declarative>

[^11]: BindingResult API (Jakarta MVC 2.0). <https://jakarta.ee/specifications/mvc/2.0/apidocs/jakarta/mvc/binding/bindingresult>

[^12]: Introduction to Jakarta EE MVC with Eclipse Krazo (Baeldung). <https://www.baeldung.com/java-ee-mvc-eclipse-krazo>

[^13]: Get to Know Jakarta MVC (Java Code Geeks). <https://www.javacodegeeks.com/2021/12/get-to-know-jakarta-mvc.html>

[^14]: JPA/Hibernate — Database-First and Code-First in Java (Medium). <https://medium.com/@a.kago1988/orm-explained-a-short-guide-to-entity-framework-and-jpa-hibernate-fe5a27de4da0>

[^15]: Top Best Practices for Spring Data JPA (Java Guides). <https://www.javaguides.net/2025/02/top-10-best-practices-for-spring-data.html>

[^16]: Spring Transaction Management: @Transactional In-Depth (Marco Behler). <https://www.marcobehler.com/guides/spring-transaction-management-transactional-in-depth>

[^17]: Integration Testing in Spring (Baeldung). <https://www.baeldung.com/integration-testing-in-spring>

[^18]: Spring MVC Test Tutorial (Petri Kainulainen). <https://www.petrikainulainen.net/spring-mvc-test-tutorial/>

[^19]: Top Real-World Examples of Successful Java Web Applications (MoldStud). <https://moldstud.com/articles/p-top-real-world-examples-of-successful-java-web-applications>

[^20]: Spring Boot RealWorld Example App (GitHub). <https://github.com/gothinkster/spring-boot-realworld-example-app>

[^21]: Develop an Enterprise Service REST API with MVC Binding using Java, Spring Boot, JPA, Hibernate (Medium). <https://medium.com/@iamsoumyadip/develop-an-enterprise-service-rest-api-with-mvc-binding-using-java-spring-boot-jpa-hibernate-3af39fd45080>

[^22]: Deep Dive into Model-View-Controller: Best Practices and Case Studies (Medium). <https://elvinbaghele.medium.com/deep-dive-into-model-view-controller-mvc-best-practices-and-case-studies-c758e13ec4cf>

[^23]: Java MVC Frameworks Comparison (JRebel by Perforce). <https://www.jrebel.com/blog/java-mvc-frameworks-comparison>

[^24]: Java EE to Spring Boot Migration Strategy (LegacyLeap). <https://legacyleap.ai/blog/java-ee-to-spring-boot-migration/>

[^25]: Secrets of DevOps in High-Traffic Java Apps (Medium). <https://medium.com/@noahblogwriter2025/secrets-of-devops-in-high-traffic-java-apps-f90794e02fb6>

[^26]: Java Virtual Machine (JVM) Performance Tuning Tutorial (Sematext). <https://sematext.com/blog/jvm-performance-tuning/>

[^27]: 11 Simple Java Performance Tuning Tips (Stackify). <https://stackify.com/java-performance-tuning/>